

01_binary_classification_epa2024

June 20, 2026

1 Classification binaire PFAS — Seuils EPA 2024 NPDWR

Objectif : Prédire si un puits californien dépasse les limites réglementaires PFAS de l'EPA 2024.

1.1 Réglementation appliquée

Composé	MCL (ng/L)	Référence
PFOA	4	EPA 2024 NPDWR
PFOS	4	EPA 2024 NPDWR
PFNA	10	EPA 2024 NPDWR
PFHxS	10	EPA 2024 NPDWR
HFPO-DA (GenX)	10	EPA 2024 NPDWR
Indice de Risque mélange	$HI > 1$	$PFNA/10 + PFHxS/10 +$ $GenX/10 + PFBS/2000$

Cible : 1 si au moins un MCL est dépassé OU si l'Indice de Risque > 1 .

1.2 Modèles comparés

- **Random Forest** (scikit-learn) — baseline interprétable, OOB score
- **XGBoost** — boosting gradient, généralement plus performant

1.3 Dataset

- Source : CA-PFAS-ASGWS.parquet — 46 338 échantillons $\times$ 201 colonnes
- Features : hydrogéologie, sol SSURGO, météo GLDAS, polluants AQS/CO, distance aux sources PFAS
- Balance : $\sim 45.7\%$ positifs (1:1.2) $\rightarrow$ quasi-équilibrée, pas besoin de suréchantillonnage fort

1.4 0. Imports et configuration

```
[1]: import warnings
import json
import pickle
from pathlib import Path

import numpy as np
import pandas as pd
```

```

import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import (
    train_test_split, StratifiedKFold, cross_validate
)
from sklearn.preprocessing import OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    roc_auc_score, average_precision_score, f1_score,
    classification_report, ConfusionMatrixDisplay,
    roc_curve, precision_recall_curve, accuracy_score,
    recall_score, precision_score, balanced_accuracy_score,
)

import xgboost as xgb

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

# Chemins
PROJECT_ROOT = Path("..")
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
MODELS_DIR = PROJECT_ROOT / "models"
FIGURES_DIR = PROJECT_ROOT / "reports" / "figures"
MODELS_DIR.mkdir(parents=True, exist_ok=True)
FIGURES_DIR.mkdir(parents=True, exist_ok=True)

# Style
plt.rcParams.update({
    "figure.dpi": 120,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.size": 11,
})
RANDOM_STATE = 42
print("Imports OK")

```

Imports OK

1.5 1. Chargement des données

```
[2]: df = pd.read_parquet(PROCESSED_DIR / "CA-PFAS-ASGWS.parquet")
      print(f"Dataset : {df.shape[0]:,} lignes × {df.shape[1]} colonnes")
      df.head(3)
```

Dataset : 46,338 lignes × 201 colonnes

```
[2]:   gm_well_id collection_date  ADONA_ngL  F53B_major_ngL  F53B_minor_ngL  \
0      100001      2019-03-28         NaN         NaN         NaN
1      100001      2020-04-28         1.0         1.0         1.0
2      100002      2019-03-28         NaN         NaN         NaN

      FTS_4_2_ngL  FTS_6_2_ngL  FTS_8_2_ngL  HFPO_DA_ngL  NEtFOSAA_ngL  ...  \
0           NaN           NaN           NaN           NaN           1.35  ...
1           NaN           NaN           NaN           2.5           1.00  ...
2           NaN           NaN           NaN           NaN           1.35  ...

      runoff_mm  soil_moi_0_10_kg_m2  soil_moi_10_40_kg_m2  \
0    0.000506         23.504955         73.092522
1    0.000722         23.421059         72.313629
2    0.000506         23.504955         73.092522

      soil_moi_40_100_kg_m2  soil_moi_100_200_kg_m2  root_zone_moist_kg_m2  \
0         149.431152         239.711273         50.487106
1         145.745392         232.140076         45.648960
2         149.431152         239.711273         50.487106

      temp_c  snowpack_mm  gldas_dist_km  soil_moisture_total_mm
0  15.603387         0.0         14.443         485.739902
1  17.293207         0.0         14.443         473.620155
2  15.603387         0.0         14.443         485.739902

[3 rows x 201 columns]
```

1.6 2. Construction de la cible EPA 2024

```
[3]: # MCLs individuels EPA 2024 NPDWR
      EPA2024_MCLS = {
          "PFOA_ngL": 4.0,
          "PFOS_ngL": 4.0,
          "PFNA_ngL": 10.0,
          "PFHxS_ngL": 10.0,
          "HFPO_DA_ngL": 10.0,
      }
      # Indice de Risque mélange (HI)
      HI_REFS = {
          "PFNA_ngL": 10.0,
```

```

    "PFHxS_ngL": 10.0,
    "HFPO_DA_ngL": 10.0,
    "PFBS_ngL": 2000.0,
}

def compute_target_epa2024(df):
    exceeded = pd.Series(False, index=df.index)
    for col, mcl in EPA2024_MCLS.items():
        if col in df.columns:
            exceeded |= df[col].fillna(0) > mcl
    hi = sum(df[c].fillna(0) / r for c, r in HI_REFS.items() if c in df.columns)
    exceeded |= hi > 1.0
    return exceeded.astype(int)

df["target"] = compute_target_epa2024(df)

n_pos = df["target"].sum()
n_neg = len(df) - n_pos
ratio = n_neg / n_pos
print(f"Positifs ( MCL) : {n_pos:,} ({100*n_pos/len(df):.1f}%)")
print(f"Négatifs (< MCL) : {n_neg:,} ({100*n_neg/len(df):.1f}%)")
print(f"Ratio négatif/positif : 1:{ratio:.2f}")

```

```

Positifs ( MCL) : 21,154 (45.7%)
Négatifs (< MCL) : 25,184 (54.3%)
Ratio négatif/positif : 1:1.19

```

```

[4]: # Distribution par composé et comparaison ancienne/nouvelle réglementation
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

# Graphe 1 : dépassements par composé
comp_stats = {}
for col, mcl in EPA2024_MCLS.items():
    n = (df[col].fillna(0) > mcl).sum()
    comp_stats[col.replace("_ngL", "")] = n
hi_exceed = (sum(df[c].fillna(0)/r for c, r in HI_REFS.items() if c in df.
    ↪columns) > 1).sum()
comp_stats["HI mélange"] = hi_exceed

keys = list(comp_stats.keys())
vals = list(comp_stats.values())
colors = ["#e74c3c" if v > 5000 else "#3498db" for v in vals]
axes[0].barh(keys, vals, color=colors)
axes[0].set_xlabel("Nombre de dépassements")
axes[0].set_title("Dépassements par composé (EPA 2024)")
axes[0].axvline(0, color="k", linewidth=0.5)
for i, v in enumerate(vals):

```

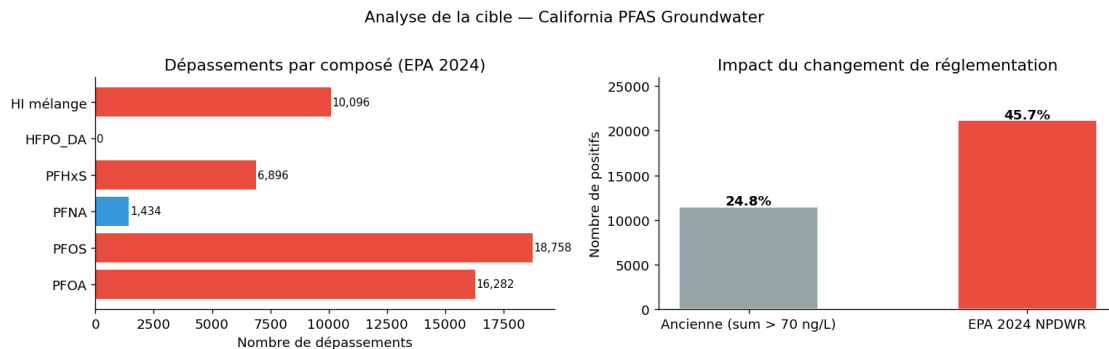
```

axes[0].text(v + 50, i, f"{v:,}", va="center", fontsize=9)

# Graphe 2 : comparaison ancienne vs nouvelle réglementation
old_target = (df["sum_pfas_ngL"].fillna(0) > 70).astype(int)
comparison = pd.DataFrame({
    "Réglementation": ["Ancienne (sum > 70 ng/L)", "EPA 2024 NPDWR"],
    "Positifs": [old_target.sum(), df["target"].sum()],
    "Taux (%)": [100*old_target.mean(), 100*df["target"].mean()],
})
bars = axes[1].bar(comparison["Réglementation"], comparison["Positifs"],
                    color=["#95a5a6", "#e74c3c"], edgecolor="white", width=0.5)
for bar, taux in zip(bars, comparison["Taux (%)"]):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 200,
                  f"{taux:.1f}%", ha="center", fontsize=11, fontweight="bold")
axes[1].set_ylabel("Nombre de positifs")
axes[1].set_title("Impact du changement de réglementation")
axes[1].set_ylim(0, 26000)

plt.suptitle("Analyse de la cible - California PFAS Groundwater", fontsize=13,
             y=1.01)
plt.tight_layout()
fig.savefig(FIGURES_DIR / "target_analysis.png", dpi=150, bbox_inches="tight")
plt.show()

```



1.7 3. Ingénierie des features

```

[5]: #
# CONFIG - protocole anti-fuite spatiale (Dong et al. 2024)
#
# DROP_LOCATION : retire les attributs de LOCALISATION PURE (lat/lon, county,
# bassins...) qui permettent au modèle de MÉMORISER la géographie plutôt que
# d'apprendre les mécanismes environnementaux.
#
# Analyse de robustesse (RF 300 arbres, dataset complet) :

```

```

#      Scénario                                #feat    AUC      F1
#      A. Tout (localisation incluse)           92    0.9745  0.918
#      B. Sans localisation pure <- ICI         86    0.9656  0.899  <- recommandé
↳ mémoire
#      C. Sans loc + sans contexte échant. 84    0.9633  0.895
#
# L\'inflation par mémorisation spatiale est réelle mais modeste (~1 pt AUC) :
# l\'essentiel du signal vient de la PROXIMITÉ AUX SOURCES (geotracker), qui
↳ est
# mécanistique et donc CONSERVÉE même en scénario B.
DROP_LOCATION = True    # <- False pour reproduire le scénario A (notebook
↳ d\'origine)

#      Features temporelles
dt = pd.to_datetime(df["collection_date"])
df["year"]    = dt.dt.year
df["month"]   = dt.dt.month
df["season"]  = ((dt.dt.month % 12) // 3 + 1).astype(int) # 1=hiver 2=printemps
↳ 3=été 4=automne

#      Colonnes à exclure - 3 types de fuite + bruit
#
#      1. Concentrations PFAS (_ngL)
#      -> Encodent directement la valeur mesurée, source de la cible
#
#      2. Indicateurs de détection binaires (_detected, label_)
#      -> "PFOA_detected=1" signifie PFOA détecté au-dessus du seuil analytique
#      corrélation avec target : PFOA_detected=0.789, PFOS_detected=0.779
#      Quand _detected=True : 89% des échantillons dépassent le MCL
#      -> Fuite directe, même nature que les concentrations
#
#      3. Ancienne cible (target_sum_gt70)
#      -> Corrélée à 0.59 avec EPA 2024 target
#
#      4. Bruit spatial : gldas_dist_km
#      -> Distance au pixel GLDAS le plus proche (max=17 km), artefact de grille
#      corr=+0.027 avec target -> aucune valeur prédictive

LEAK_SUFFIXES = ("_ngL",)
LEAK_BOOL_SUFFIXES = ("_detected",) # indicateurs de détection PFAS
LEAK_PREFIXES = ("label_",)
LEAK_EXACT = {
    "sum_pfas_ngL", "pfas_class_assignment", "gm_well_id",
    "collection_date", "target", "target_sum_gt70",
    "gldas_dist_km", # artefact de grille GLDAS, corr=0.027
}

```

```

# Colonnes >90% null - imputation médiane ne suffira pas, trop de signal
↳ artificiel
HIGH_NULL_DROP = {
    "cocontam_xylenes",          # 100% null
    "cocontam_tmb124",          # 99% null
    "cocontam_dce12c",          # 99% null
    "cocontam_btbzt",           # 99% null
    "soil_gradation_uniformity", # 98% null (CA manque siltco_r)
    "soil_gradation_curvature",  # 98% null
    "soil_silt_coarse_pct",       # 95% null
    "soil_silt_fine_pct",         # 95% null
    "cocontam_no3n",             # 95% null
    "well_depth_ft",             # 95% null (non renseigné GAMA)
}

# Identifiants géographiques redondants (déjà encodés par county/regional_board)
DROP_REDUNDANT = {"dwr_basin", "sgma_basin_name", "sgma_subbasin_name"}

# Localisation PURE (protocole Dong et al.) - retirée si DROP_LOCATION
# Conserve volontairement les features de PROXIMITÉ aux sources (geotracker) :
# elles sont mécanistiques (transport depuis une source), pas positionnelles.
LOCATION_FEATURES = {
    "latitude", "longitude",
    "county", "regional_board", "dwr_region", "sgma_region_office",
}
loc_drop = LOCATION_FEATURES if DROP_LOCATION else set()

# Catégorielles sélectionnées
CATEGORICAL_FEATURES = [
    "gm_well_category",          # Type de puits : Municipal/Monitoring/Domestic...
    "nearest_geotracker_type",   # Type du site PFAS le plus proche (industrie,
    ↳ armée, feu...)
    "soil_texture_class",        # Classe texturale USDA (proxy perméabilité)
    "sgma_region_office",        # Bureau régional SGMA (basin adjudication)
    "gm_dataset_name",          # Programme surveillance GAMA
    "county",                    # Comté (58 uniques - proxy land-use/industrial)
    "regional_board",           # Tableau régional eau (9 boards)
    "dwr_region",               # Région DWR (10 régions)
]
cat_cols = [c for c in CATEGORICAL_FEATURES if c in df.columns and c not in
↳ loc_drop]

# Numériques
num_cols = [
    c for c in df.columns
    if df[c].dtype.kind in ("f", "i", "u")
    and not any(c.endswith(s) for s in LEAK_SUFFIXES)
]

```

```

    and not any(c.endswith(s) for s in LEAK_BOOL_SUFFIXES)
    and not any(c.startswith(p) for p in LEAK_PREFIXES)
    and c not in LEAK_EXACT
    and c not in HIGH_NULL_DROP
    and c not in DROP_REDUNDANT
    and c not in loc_drop
    and c not in cat_cols
]

feature_names = num_cols + cat_cols

# Résumé des exclusions
n_detected = sum(1 for c in df.columns if c.endswith("_detected"))
n_label = sum(1 for c in df.columns if c.startswith("label_"))
n_ngL = sum(1 for c in df.columns if c.endswith("_ngL"))

print(" ")
print(f" Exclusions (fuite + bruit) | DROP_LOCATION = {DROP_LOCATION}")
print(" ")
print(f" _ngL (concentrations) : {n_ngL} colonnes")
print(f" label_ (détection PFAS) : {n_label} colonnes")
print(f" _detected (détection PFAS) : {n_detected} colonnes <- corr_
↳ jusqu'à 0.79")
print(f" target_sum_gt70 : 1 colonne <- ancienne cible (corr=0.
↳ 59)")
print(f" gldas_dist_km : 1 colonne <- bruit artefact grille")
print(f" >90% null : {len(HIGH_NULL_DROP)} colonnes")
print(f" redondantes géo : {len(DROP_REDUNDANT)} colonnes")
if DROP_LOCATION:
    print(f" localisation pure (Dong) : {len(LOCATION_FEATURES)} colonnes
↳ <- {'', '.join(sorted(LOCATION_FEATURES))}")
print(" ")
print(f" Features numériques retenues : {len(num_cols)}")
print(f" Features catégorielles : {len(cat_cols)}")
print(f" TOTAL : {len(feature_names)}")
print(" ")
print()
print("Groupes de features retenues :")
groups = {
    "SPATIAL": [c for c in num_cols if c in ["latitude", "longitude"]],
    "GEOTRACKER": [c for c in num_cols if "geotracker" in c],
    "GLDAS_HYDRO": [c for c in num_cols if any(x in c for x in
↳ ["rainfall", "et_mm", "runoff", "soil_moi", "root_zone", "temp_c", "snowpack", "soil_moisture_tota
    "SSURGO_SOL": [c for c in num_cols if c.startswith("soil_") and c not_
↳ in_
↳ ["soil_moi_0_10_kg_m2", "soil_moi_10_40_kg_m2", "soil_moi_40_100_kg_m2", "soil_moi_100_200_kg_
    "AQS_AIR": [c for c in num_cols if c.startswith("aqs_")],

```



```

"CO_CONTAM":      [c for c in num_cols if c.startswith("cocontam_")],
"TEMPOREL":       [c for c in num_cols if c in ["year", "month", "season"]],
"CATEGORIEL":     cat_cols,
}
for gname, gcols in groups.items():
    print(f"  {gname:<15} {len(gcols):3d} features")

X = df[feature_names].copy()
y = df["target"].values
print(f"\nMatrice X : {X.shape}")

```

```

Exclusions (fuite + bruit) | DROP_LOCATION = True

_ngL   (concentrations)      : 32 colonnes
label_ (détection PFAS)      : 31 colonnes
_detected (détection PFAS)   : 31 colonnes  <- corr jusqu'à 0.79
target_sum_gt70              : 1 colonne    <- ancienne cible (corr=0.59)
gldas_dist_km                : 1 colonne    <- bruit artefact grille
>90% null                    : 10 colonnes
redondantes géo              : 3 colonnes
localisation pure (Dong)      : 6 colonnes  <- county, dwr_region, latitude,
longitude, regional_board, sgma_region_office

Features numériques retenues : 82
Features catégorielles       : 4
TOTAL                        : 86

Groupes de features retenues :
SPATIAL           0 features
GEOTRACKER        5 features
GLDAS_HYDRO       11 features
SSURGO_SOL        16 features
AQS_AIR           8 features
CO_CONTAM         39 features
TEMPOREL          3 features
CATEGORIEL        4 features

```

Matrice X : (46338, 86)

```

[6]: # Aperçu des taux de valeurs manquantes par groupe de features
missing = X.isna().mean().sort_values(ascending=False)
missing_top = missing[missing > 0].head(30)

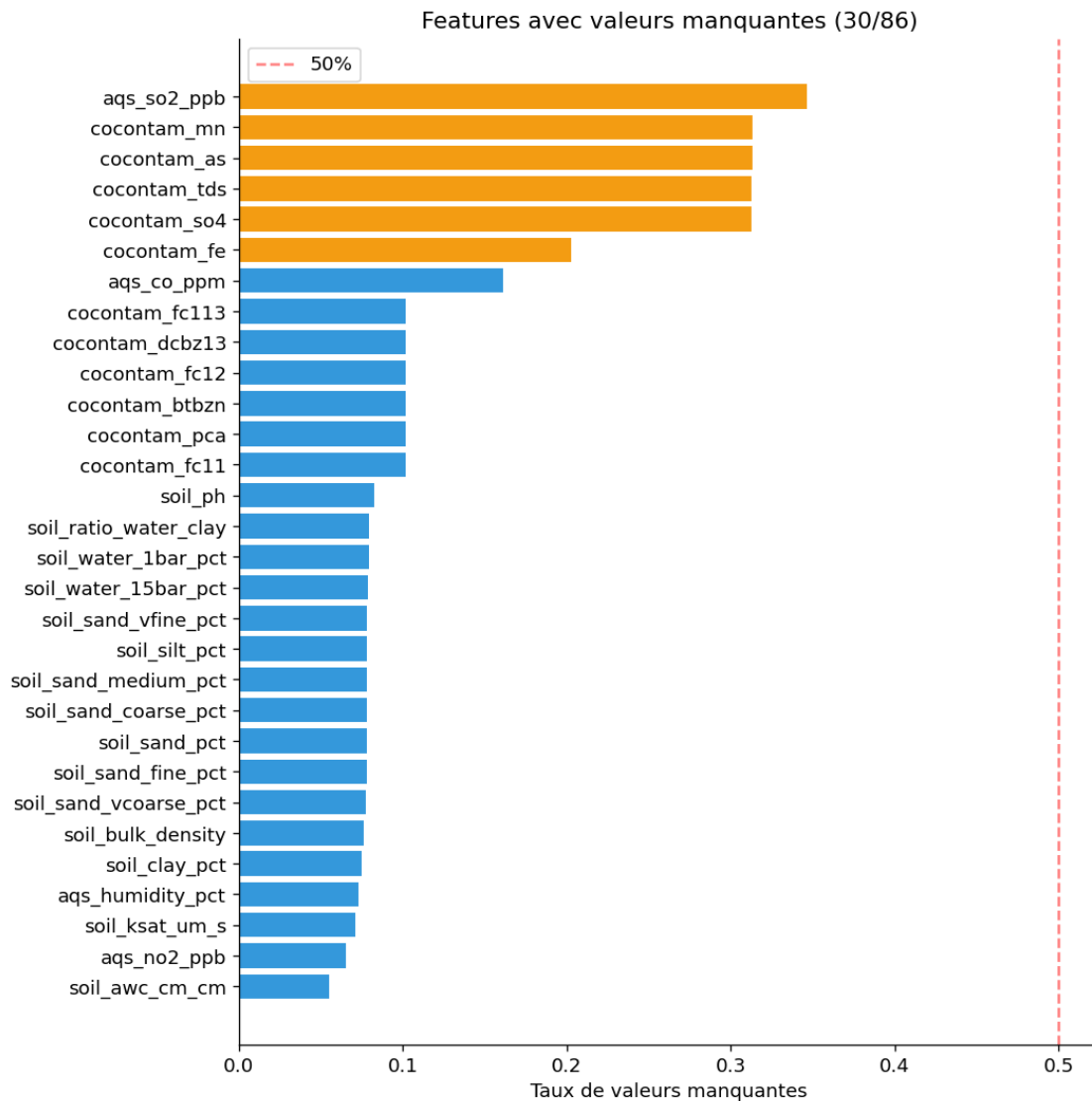
if len(missing_top) > 0:
    fig, ax = plt.subplots(figsize=(9, max(4, len(missing_top)*0.3)))

```

```

    colors = ["#e74c3c" if v > 0.5 else "#f39c12" if v > 0.2 else "#3498db" for
↪v in missing_top]
    ax.barh(missing_top.index[::-1], missing_top.values[::-1], color=colors[::
↪-1])
    ax.set_xlabel("Taux de valeurs manquantes")
    ax.set_title(f"Features avec valeurs manquantes ({len(missing_top)}/
↪{len(feature_names)})")
    ax.axvline(0.5, color="red", linestyle="--", alpha=0.5, label="50%")
    ax.legend()
    plt.tight_layout()
    plt.show()
else:
    print("Aucune valeur manquante dans les features !")

```



1.8 4. Prétraitement et split train/test

```
[7]: # Split stratifié 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)
print(f"Train : {len(y_train):,} ({100*y_train.mean():.1f}% positifs)")
print(f"Test  : {len(y_test):,}  ({100*y_test.mean():.1f}% positifs)")

# Préprocesseur sklearn
numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
])
categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
    ("encoder", OrdinalEncoder(
        handle_unknown="use_encoded_value",
        unknown_value=-1,
        encoded_missing_value=-1,
    )),
])
preprocessor = ColumnTransformer([
    ("num", numeric_pipe, num_cols),
    ("cat", categorical_pipe, cat_cols),
], remainder="drop")

X_train_proc = preprocessor.fit_transform(X_train)
X_test_proc = preprocessor.transform(X_test)
print(f"\nX_train_proc : {X_train_proc.shape}")
```

```
Train : 37,070 (45.7% positifs)
Test  : 9,268 (45.7% positifs)
```

```
X_train_proc : (37070, 86)
```

1.9 5. Fonctions utilitaires (métriques + graphiques)

```
[8]: def evaluate_model(name, y_true, y_pred, y_proba):
    """Calcule et affiche les métriques principales.

    Le RAPPEL (recall) de la classe positive mesure la part de puits réellement
    au-dessus des MCL que le modèle détecte - métrique prioritaire en contexte
    sanitaire (un faux négatif = puits contaminé non signalé). La balanced
    accuracy (moyenne des rappels par classe) est insensible au déséquilibre.
    """
```

```

auc = roc_auc_score(y_true, y_proba)
ap = average_precision_score(y_true, y_proba)
f1 = f1_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)          # rappel classe positive
↳(sensibilité)
prec = precision_score(y_true, y_pred)      # précision classe positive
acc = accuracy_score(y_true, y_pred)
bacc = balanced_accuracy_score(y_true, y_pred)
rep = classification_report(y_true, y_pred, target_names=["< MCL (0)", "
↳MCL (1)"])
print(f"{' '*55}")
print(f"  {name}")
print(f"{' '*55}")
print(f"  ROC-AUC          : {auc:.4f}")
print(f"  Average Precision : {ap:.4f}")
print(f"  Rappel ( MCL)      : {rec:.4f}  ← détection des positifs")
print(f"  Précision ( MCL)  : {prec:.4f}")
print(f"  F1 ( MCL)         : {f1:.4f}")
print(f"  Accuracy          : {acc:.4f}")
print(f"  Balanced Accuracy  : {bacc:.4f}  ← insensible au déséquilibre")
print(f"{' '*55}")
print(rep)
return {"name": name, "roc_auc": auc, "avg_precision": ap, "f1": f1,
        "recall": rec, "precision": prec, "accuracy": acc,
        "balanced_accuracy": bacc}

def plot_roc_pr(models_results, y_test, title=""):
    """Courbes ROC et Précision-Rappel superposées pour plusieurs modèles."""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
    palette = ["#2980b9", "#e74c3c", "#27ae60", "#8e44ad"]

    for i, (name, y_proba, metrics) in enumerate(models_results):
        color = palette[i % len(palette)]
        # ROC
        fpr, tpr, _ = roc_curve(y_test, y_proba)
        ax1.plot(fpr, tpr, lw=2, color=color,
                label=f"{name} (AUC={metrics['roc_auc']:.4f})")
        # PR
        prec, rec, _ = precision_recall_curve(y_test, y_proba)
        ax2.plot(rec, prec, lw=2, color=color,
                label=f"{name} (AP={metrics['avg_precision']:.4f})")

    baseline_prev = y_test.mean()
    ax1.plot([0,1],[0,1], "k--", lw=1, label="Aléatoire")
    ax2.axhline(baseline_prev, color="k", linestyle="--", lw=1,
                label=f"Baseline ({baseline_prev:.2f})")

```

```

ax1.set(xlabel="Taux faux positifs", ylabel="Taux vrais positifs",
        title="Courbe ROC")
ax2.set(xlabel="Rappel", ylabel="Précision",
        title="Courbe Précision-Rappel")
ax1.legend(fontsize=9); ax2.legend(fontsize=9)
ax1.grid(alpha=0.3); ax2.grid(alpha=0.3)

if title:
    fig.suptitle(title, fontsize=13)
plt.tight_layout()
fig.savefig(FIGURES_DIR / "ml_binary_roc_pr.png", dpi=150,
            bbox_inches="tight")
plt.show()

def plot_confusion_matrices(models_preds, y_test):
    """Matrices de confusion côte à côte."""
    n = len(models_preds)
    fig, axes = plt.subplots(1, n, figsize=(5*n, 4))
    if n == 1: axes = [axes]
    for ax, (name, y_pred) in zip(axes, models_preds):
        ConfusionMatrixDisplay.from_predictions(
            y_test, y_pred,
            display_labels=["< MCL (0)", " MCL (1)"],
            cmap="Blues", ax=ax, colorbar=False,
        )
        ax.set_title(f"Matrice de confusion\n{name}")
    plt.tight_layout()
    fig.savefig(FIGURES_DIR / "ml_binary_confusion_matrices.png", dpi=150,
                bbox_inches="tight")
    plt.show()

print("Fonctions utilitaires prêtes.")

```

Fonctions utilitaires prêtes.

1.10 6. Random Forest

```

[9]: # Paramètres RF
RF_PARAMS = {
    "n_estimators": 500,
    "max_features": "sqrt",
    "min_samples_leaf": 2,
    "max_depth": None,
    "oob_score": True,
    "class_weight": "balanced", # compense le déséquilibre résiduel
    "n_jobs": -1,
    "random_state": RANDOM_STATE,
}

```

```

}

print("Entraînement Random Forest (500 arbres)...")
rf = RandomForestClassifier(**RF_PARAMS)
rf.fit(X_train_proc, y_train)
print(f"OOB score : {rf.oob_score_:.4f}")

```

Entraînement Random Forest (500 arbres)...

OOB score : 0.9058

```

[10]: # Évaluation RF
y_pred_rf = rf.predict(X_test_proc)
y_proba_rf = rf.predict_proba(X_test_proc)[: , 1]
metrics_rf = evaluate_model("Random Forest", y_test, y_pred_rf, y_proba_rf)

```

Random Forest

```

ROC-AUC          : 0.9659
Average Precision : 0.9613
Rappel ( MCL)    : 0.8934  ← détection des positifs
Précision ( MCL) : 0.9056
F1 ( MCL)        : 0.8995
Accuracy         : 0.9088
Balanced Accuracy : 0.9076  ← insensible au déséquilibre

```

	precision	recall	f1-score	support
< MCL (0)	0.91	0.92	0.92	5037
MCL (1)	0.91	0.89	0.90	4231
accuracy			0.91	9268
macro avg	0.91	0.91	0.91	9268
weighted avg	0.91	0.91	0.91	9268

```

[11]: # Validation croisée RF
print("Validation croisée 5-fold RF...")
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
cv_rf = cross_validate(
    RandomForestClassifier(**{**RF_PARAMS, "oob_score": False}),
    X_train_proc, y_train,
    cv=skf,
    scoring=["roc_auc", "average_precision", "f1", "recall",
    ↪ "balanced_accuracy"],
    n_jobs=-1,
)

```

```

for metric in ["roc_auc", "average_precision", "f1", "recall",
↪ "balanced_accuracy"]:
    scores = cv_rf[f"test_{metric}"]
    print(f" {metric:25s}: {scores.mean():.4f} ± {scores.std():.4f}")

```

Validation croisée 5-fold RF...

```

roc_auc           : 0.9619 ± 0.0010
average_precision : 0.9575 ± 0.0015
f1                : 0.8913 ± 0.0037
recall            : 0.8865 ± 0.0022
balanced_accuracy : 0.9001 ± 0.0034

```

1.11 7. XGBoost

```

[12]: # Paramètres XGBoost
# scale_pos_weight = n_neg / n_pos pour compenser le déséquilibre de classes
scale_pos = (y_train == 0).sum() / (y_train == 1).sum()
print(f"scale_pos_weight = {scale_pos:.4f}")

XGB_PARAMS = {
    "n_estimators": 1000,
    "learning_rate": 0.05,
    "max_depth": 6,
    "subsample": 0.8,
    "colsample_bytree": 0.8,
    "min_child_weight": 3,
    "gamma": 0.1,
    "reg_alpha": 0.1, # L1
    "reg_lambda": 1.0, # L2
    "scale_pos_weight": scale_pos,
    "tree_method": "hist", # rapide sur CPU
    "n_jobs": -1,
    "random_state": RANDOM_STATE,
    "eval_metric": "auc",
}

print("Entraînement XGBoost (1000 rounds, early stopping)...")

# Split interne pour early stopping (20% du train)
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train_proc, y_train, test_size=0.15, stratify=y_train,
↪ random_state=RANDOM_STATE
)

xgb_model = xgb.XGBClassifier(
    **XGB_PARAMS,
    early_stopping_rounds=50,

```

```

        verbosity=0,
    )
    xgb_model.fit(
        X_tr, y_tr,
        eval_set=[(X_val, y_val)],
        verbose=False,
    )
    print(f"Best iteration : {xgb_model.best_iteration}")
    print(f"Best AUC (val) : {xgb_model.best_score:.4f}")

```

scale_pos_weight = 1.1905

Entraînement XGBoost (1000 rounds, early stopping)...

Best iteration : 999

Best AUC (val) : 0.9675

```

[13]: # Évaluation XGBoost
y_pred_xgb = xgb_model.predict(X_test_proc)
y_proba_xgb = xgb_model.predict_proba(X_test_proc)[:, 1]
metrics_xgb = evaluate_model("XGBoost", y_test, y_pred_xgb, y_proba_xgb)

```

XGBoost

```

ROC-AUC          : 0.9665
Average Precision : 0.9616
Rappel ( MCL)    : 0.9062  ← détection des positifs
Précision ( MCL) : 0.8985
F1 ( MCL)        : 0.9023
Accuracy         : 0.9104
Balanced Accuracy : 0.9101  ← insensible au déséquilibre

```

	precision	recall	f1-score	support
< MCL (0)	0.92	0.91	0.92	5037
MCL (1)	0.90	0.91	0.90	4231
accuracy			0.91	9268
macro avg	0.91	0.91	0.91	9268
weighted avg	0.91	0.91	0.91	9268

```

[14]: # Validation croisée XGBoost
print("Validation croisée 5-fold XGBoost...")
cv_xgb = cross_validate(
    xgb.XGBClassifier(
        **{k: v for k, v in XGB_PARAMS.items() if k != "eval_metric"},
        early_stopping_rounds=None,
        verbosity=0,
    )

```



```

    ),
    X_train_proc, y_train,
    cv=skf,
    scoring=["roc_auc", "average_precision", "f1", "recall",
    ↪ "balanced_accuracy"],
    n_jobs=-1,
)
for metric in ["roc_auc", "average_precision", "f1", "recall",
    ↪ "balanced_accuracy"]:
    scores = cv_xgb[f"test_{metric}"]
    print(f" {metric:25s}: {scores.mean():.4f} ± {scores.std():.4f}")

```

Validation croisée 5-fold XGBoost...

```

roc_auc           : 0.9647 ± 0.0010
average_precision : 0.9593 ± 0.0010
f1                : 0.8976 ± 0.0019
recall            : 0.8998 ± 0.0036
balanced_accuracy : 0.9057 ± 0.0018

```

1.12 8. Optimisation des hyperparamètres — Optuna

Recherche bayésienne (TPE sampler) pour RF et XGBoost. - **RF** : 3-fold CV, 40 trials (~10 min)
- **XGBoost** : validation interne + early stopping, 60 trials (~8 min)

Mettez `RUN_OPTUNA = False` pour sauter et garder les paramètres par défaut.

```

[15]: import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)  # supprimer le spam

RUN_OPTUNA = True  # ← mettre False pour passer cette section
N_TRIALS_RF = 40
N_TRIALS_XGB = 60
TIMEOUT_RF = 600  # secondes max par study (10 min)
TIMEOUT_XGB = 480  # secondes max par study (8 min)

print(f"Optuna {optuna.__version__}")
print(f"Trials RF={N_TRIALS_RF} (timeout={TIMEOUT_RF}s) | XGB={N_TRIALS_XGB}
    ↪ (timeout={TIMEOUT_XGB}s)")

```

Optuna 4.9.0

Trials RF=40 (timeout=600s) | XGB=60 (timeout=480s)

```

[16]: # Optuna RF
if RUN_OPTUNA:
    from sklearn.model_selection import cross_val_score

    def objective_rf(trial):
        params = {

```

```

        "n_estimators":      trial.suggest_int("n_estimators", 200, 800,
↪step=100),
        "max_depth":        trial.suggest_categorical("max_depth", [None,
↪15, 20, 30]),
        "max_features":     trial.suggest_categorical("max_features",
↪["sqrt", "log2", 0.3, 0.5]),
        "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 10),
        "min_samples_split":trial.suggest_int("min_samples_split", 2, 15),
        "class_weight":     "balanced",
        "oob_score":        False,
        "n_jobs":           -1,
        "random_state":     RANDOM_STATE,
    }
    model = RandomForestClassifier(**params)
    scores = cross_val_score(
        model, X_train_proc, y_train,
        cv=StratifiedKFold(3, shuffle=True, random_state=RANDOM_STATE),
        scoring="roc_auc", n_jobs=1,  # n_jobs=1 car RF est déjà //
    )
    return scores.mean()

sampler_rf = optuna.samplers.TPESampler(seed=RANDOM_STATE)
study_rf = optuna.create_study(direction="maximize", sampler=sampler_rf,
                               study_name="rf_epa2024")
# Démarrer avec les paramètres actuels comme point de départ
study_rf.enqueue_trial({
    "n_estimators": 500, "max_depth": None, "max_features": "sqrt",
    "min_samples_leaf": 2, "min_samples_split": 2,
})

study_rf.optimize(objective_rf, n_trials=N_TRIALS_RF,
                  timeout=TIMEOUT_RF, show_progress_bar=True)

print(f"\n[RF] Meilleur AUC-CV : {study_rf.best_value:.4f}")
print(f"[RF] Meilleurs params :")
for k, v in study_rf.best_params.items():
    print(f"    {k:<25} = {v}")

# Visualisation de l'historique
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

values = [t.value for t in study_rf.trials if t.value is not None]
best_so_far = [max(values[:i+1]) for i in range(len(values))]
axes[0].plot(values, "o", alpha=0.4, color="steelblue", ms=5, label="Trial")
axes[0].plot(best_so_far, "-", color="navy", lw=2, label="Meilleur")
axes[0].set_xlabel("Trial")
axes[0].set_ylabel("AUC-CV (3-fold)")

```

```

axes[0].set_title("Random Forest - Historique Optuna")
axes[0].legend()
axes[0].grid(alpha=0.3)

# Importance des hyperparamètres
try:
    imp = optuna.importance.get_param_importances(study_rf)
    axes[1].barh(list(imp.keys())[:-1], list(imp.values())[:-1],
↳color="steelblue")
    axes[1].set_xlabel("Importance relative")
    axes[1].set_title("RF - Importance des hyperparamètres")
    axes[1].grid(axis="x", alpha=0.3)
except Exception:
    axes[1].text(0.5, 0.5, "Pas assez de trials", ha="center",
↳transform=axes[1].transAxes)

plt.tight_layout()
plt.savefig(FIGURES_DIR / "optuna_rf_history.png", dpi=130,
↳bbox_inches="tight")
plt.show()
else:
    print("Optuna RF ignoré (RUN_OPTUNA=False)")
    study_rf = None

```

0% | 0/40 [00:00<?, ?it/s]

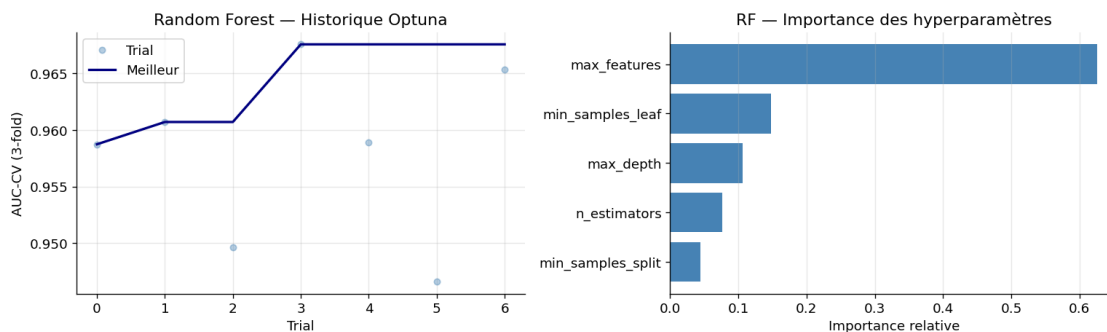
[RF] Meilleur AUC-CV : 0.9676

[RF] Meilleurs params :

```

n_estimators      = 400
max_depth         = 20
max_features      = 0.5
min_samples_leaf  = 2
min_samples_split = 2

```



```

[17]: # Optuna XGBoost
if RUN_OPTUNA:
    # Split interne stable pour tous les trials
    X_opt_tr, X_opt_val, y_opt_tr, y_opt_val = train_test_split(
        X_train_proc, y_train, test_size=0.18,
        stratify=y_train, random_state=RANDOM_STATE,
    )

    def objective_xgb(trial):
        params = {
            "n_estimators": trial.suggest_int("n_estimators", 200, 2000, ↵
↵step=100),
            "learning_rate": trial.suggest_float("learning_rate", 0.005, 0.
↵3, log=True),
            "max_depth": trial.suggest_int("max_depth", 3, 12),
            "subsample": trial.suggest_float("subsample", 0.5, 1.0),
            "colsample_bytree": trial.suggest_float("colsample_bytree", 0.4, 1.
↵0),
            "colsample_bylevel": trial.suggest_float("colsample_bylevel", 0.4, ↵
↵1.0),
            "min_child_weight": trial.suggest_int("min_child_weight", 1, 15),
            "gamma": trial.suggest_float("gamma", 0.0, 2.0),
            "reg_alpha": trial.suggest_float("reg_alpha", 1e-8, 10.0, ↵
↵log=True),
            "reg_lambda": trial.suggest_float("reg_lambda", 1e-8, 10.0, ↵
↵log=True),
            "scale_pos_weight": scale_pos,
            "tree_method": "hist",
            "n_jobs": -1,
            "random_state": RANDOM_STATE,
            "eval_metric": "auc",
            "verbosity": 0,
        }
        model = xgb.XGBClassifier(**params, early_stopping_rounds=40)
        model.fit(X_opt_tr, y_opt_tr,
            eval_set=[(X_opt_val, y_opt_val)],
            verbose=False)
        return roc_auc_score(y_opt_val, model.predict_proba(X_opt_val)[: , 1])

    sampler_xgb = optuna.samplers.TPESampler(seed=RANDOM_STATE)
    pruner_xgb = optuna.pruners.MedianPruner(n_startup_trials=10, ↵
↵n_warmup_steps=5)
    study_xgb = optuna.create_study(direction="maximize",
                                    sampler=sampler_xgb, pruner=pruner_xgb,
                                    study_name="xgb_epa2024")

    # Point de départ = paramètres actuels

```

```

study_xgb.enqueue_trial({
    "n_estimators": 1000, "learning_rate": 0.05, "max_depth": 6,
    "subsample": 0.8, "colsample_bytree": 0.8, "colsample_bylevel": 1.0,
    "min_child_weight": 3, "gamma": 0.1,
    "reg_alpha": 0.1, "reg_lambda": 1.0,
})

study_xgb.optimize(objective_xgb, n_trials=N_TRIALS_XGB,
                   timeout=TIMEOUT_XGB, show_progress_bar=True)

print(f"\n[XGB] Meilleur AUC-val : {study_xgb.best_value:.4f}")
print(f"[XGB] Meilleurs params :")
for k, v in study_xgb.best_params.items():
    print(f"    {k:<25} = {v}")

# Visualisation
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

values_x = [t.value for t in study_xgb.trials if t.value is not None]
best_so_far_x = [max(values_x[:i+1]) for i in range(len(values_x))]
axes[0].plot(values_x, "o", alpha=0.4, color="crimson", ms=5, label="Trial")
axes[0].plot(best_so_far_x, "-", color="darkred", lw=2, label="Meilleur")
axes[0].set_xlabel("Trial")
axes[0].set_ylabel("AUC val interne")
axes[0].set_title("XGBoost - Historique Optuna")
axes[0].legend()
axes[0].grid(alpha=0.3)

try:
    imp_x = optuna.importance.get_param_importances(study_xgb)
    axes[1].barh(list(imp_x.keys())[::-1], list(imp_x.values())[::-1],
    color="crimson")
    axes[1].set_xlabel("Importance relative")
    axes[1].set_title("XGBoost - Importance des hyperparamètres")
    axes[1].grid(axis="x", alpha=0.3)
except Exception:
    axes[1].text(0.5, 0.5, "Pas assez de trials", ha="center",
    transform=axes[1].transAxes)

plt.tight_layout()
plt.savefig(FIGURES_DIR / "optuna_xgb_history.png", dpi=130,
    bbox_inches="tight")
plt.show()
else:
    print("Optuna XGBoost ignoré (RUN_OPTUNA=False)")
    study_xgb = None

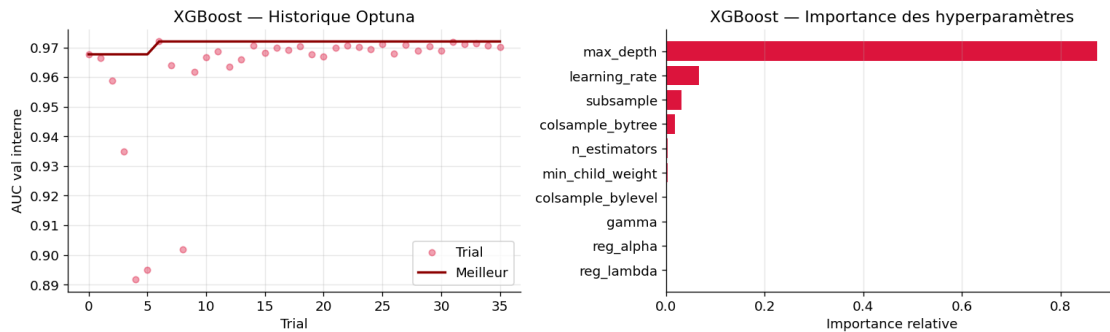
```

0%| | 0/60 [00:00<?, ?it/s]

[XGB] Meilleur AUC-val : 0.9720

[XGB] Meilleurs params :

```
n_estimators          = 2000
learning_rate          = 0.11947399979512872
max_depth              = 12
subsample              = 0.9474136752138245
colsample_bytree       = 0.7587399872866512
colsample_bylevel      = 0.9531245410138701
min_child_weight       = 2
gamma                  = 0.3919657248382904
reg_alpha              = 2.5529693461039728e-08
reg_lambda             = 8.471746987003668e-06
```



```
[18]: # Ré-entraînement avec les meilleurs hyperparamètres Optuna
if RUN_OPTUNA and study_rf is not None and study_xgb is not None:

    # Random Forest
    best_rf_params = {
        **study_rf.best_params,
        "oob_score": True,
        "class_weight": "balanced",
        "n_jobs": -1,
        "random_state": RANDOM_STATE,
    }

    print("Ré-entraînement RF avec meilleurs params Optuna...")
    rf = RandomForestClassifier(**best_rf_params)
    rf.fit(X_train_proc, y_train)
    print(f" OOB score : {rf.oob_score_:.4f}")

    y_pred_rf = rf.predict(X_test_proc)
    y_proba_rf = rf.predict_proba(X_test_proc)[: , 1]
    metrics_rf = evaluate_model("RF (Optuna)", y_test, y_pred_rf, y_proba_rf)
```

```

# XGBoost
best_xgb_params = {
    **study_xgb.best_params,
    "scale_pos_weight": scale_pos,
    "tree_method": "hist",
    "n_jobs": -1,
    "random_state": RANDOM_STATE,
    "eval_metric": "auc",
    "verbosity": 0,
}

print("\nRé-entraînement XGBoost avec meilleurs params Optuna...")
xgb_model = xgb.XGBClassifier(**best_xgb_params, early_stopping_rounds=50)
xgb_model.fit(
    X_tr, y_tr,
    eval_set=[(X_val, y_val)],
    verbose=False,
)

print(f" Best iteration : {xgb_model.best_iteration}")

y_pred_xgb = xgb_model.predict(X_test_proc)
y_proba_xgb = xgb_model.predict_proba(X_test_proc)[: , 1]
metrics_xgb = evaluate_model("XGB (Optuna)", y_test, y_pred_xgb,
↪ y_proba_xgb)

# Gain vs baseline
print("\n")
print(" Gain Optuna vs paramètres par défaut ")
print(" ")
print(" Modèle AUC avant AUC après Δ ")
print(" ")
# récupérer les scores baseline depuis le dict comparison (cellule suivante)
# on les réaffiche simplement ici
auc_rf_optuna = metrics_rf["roc_auc"]
auc_xgb_optuna = metrics_xgb["roc_auc"]
print(f" RF - {auc_rf_optuna:.4f} - ")
↪ " )
print(f" XGBoost - {auc_xgb_optuna:.4f} - ")
↪ " )
print(" ")
print("\n (les valeurs 'avant' sont dans la cellule 6/7 au-dessus)")

else:
    print("Modèles non ré-entraînés - Optuna était désactivé.")
    print("RF et XGBoost gardent les paramètres par défaut des sections 6 et 7.
↪ ")

```

Ré-entraînement RF avec meilleurs params Optuna...

OOB score : 0.9239

RF (Optuna)

ROC-AUC : 0.9740
Average Precision : 0.9690
Rappel (MCL) : 0.9152 ← détection des positifs
Précision (MCL) : 0.9245
F1 (MCL) : 0.9198
Accuracy : 0.9272
Balanced Accuracy : 0.9262 ← insensible au déséquilibre

	precision	recall	f1-score	support
< MCL (0)	0.93	0.94	0.93	5037
MCL (1)	0.92	0.92	0.92	4231
accuracy			0.93	9268
macro avg	0.93	0.93	0.93	9268
weighted avg	0.93	0.93	0.93	9268

Ré-entraînement XGBoost avec meilleurs params Optuna...

Best iteration : 164

XGB (Optuna)

ROC-AUC : 0.9708
Average Precision : 0.9666
Rappel (MCL) : 0.9156 ← détection des positifs
Précision (MCL) : 0.9085
F1 (MCL) : 0.9121
Accuracy : 0.9194
Balanced Accuracy : 0.9191 ← insensible au déséquilibre

	precision	recall	f1-score	support
< MCL (0)	0.93	0.92	0.93	5037
MCL (1)	0.91	0.92	0.91	4231
accuracy			0.92	9268
macro avg	0.92	0.92	0.92	9268
weighted avg	0.92	0.92	0.92	9268

Gain Optuna vs paramètres par défaut

Modèle	AUC avant	AUC après	Δ
RF	-	0.9740	-
XGBoost	-	0.9708	-

(les valeurs 'avant' sont dans la cellule 6/7 au-dessus)

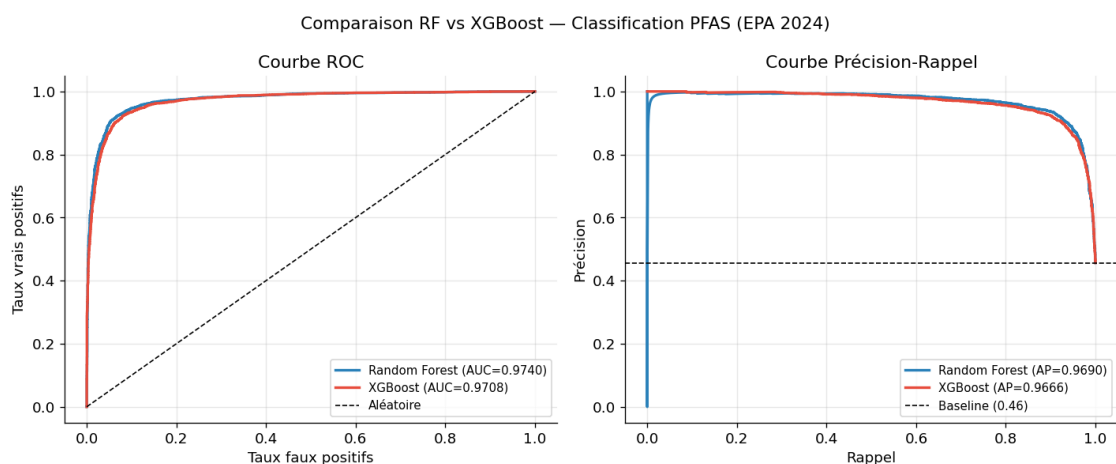
1.13 8. Comparaison des modèles

```
[19]: # Tableau comparatif
comparison_df = pd.DataFrame([
    {
        "Modèle": "Random Forest",
        "ROC-AUC (test)": metrics_rf["roc_auc"],
        "AP (test)": metrics_rf["avg_precision"],
        "Rappel (test)": metrics_rf["recall"],
        "Précision (test)": metrics_rf["precision"],
        "F1 (test)": metrics_rf["f1"],
        "Bal.Acc (test)": metrics_rf["balanced_accuracy"],
        "Accuracy (test)": metrics_rf["accuracy"],
        "ROC-AUC (CV)": f"{cv_rf['test_roc_auc'].mean():.
↪4f}±{cv_rf['test_roc_auc'].std():.4f}",
        "Rappel (CV)": f"{cv_rf['test_recall'].mean():.
↪4f}±{cv_rf['test_recall'].std():.4f}",
    },
    {
        "Modèle": "XGBoost",
        "ROC-AUC (test)": metrics_xgb["roc_auc"],
        "AP (test)": metrics_xgb["avg_precision"],
        "Rappel (test)": metrics_xgb["recall"],
        "Précision (test)": metrics_xgb["precision"],
        "F1 (test)": metrics_xgb["f1"],
        "Bal.Acc (test)": metrics_xgb["balanced_accuracy"],
        "Accuracy (test)": metrics_xgb["accuracy"],
        "ROC-AUC (CV)": f"{cv_xgb['test_roc_auc'].mean():.
↪4f}±{cv_xgb['test_roc_auc'].std():.4f}",
        "Rappel (CV)": f"{cv_xgb['test_recall'].mean():.
↪4f}±{cv_xgb['test_recall'].std():.4f}",
    },
])
comparison_df.set_index("Modèle", inplace=True)
```

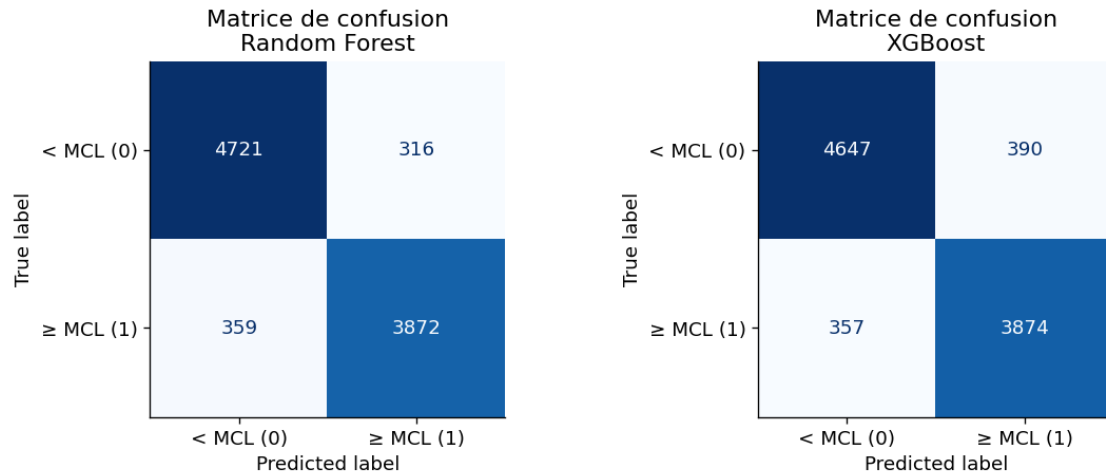
```
comparison_df.style.background_gradient(cmap="RdYlGn", subset=["ROC-AUC",
    ↪(test)", "AP (test)", "Rappel (test)", "Précision (test)", "F1 (test)", "Bal.
    ↪Acc (test)", "Accuracy (test)"])
```

[19]: <pandas.io.formats.style.Styler at 0x7cd9186ac800>

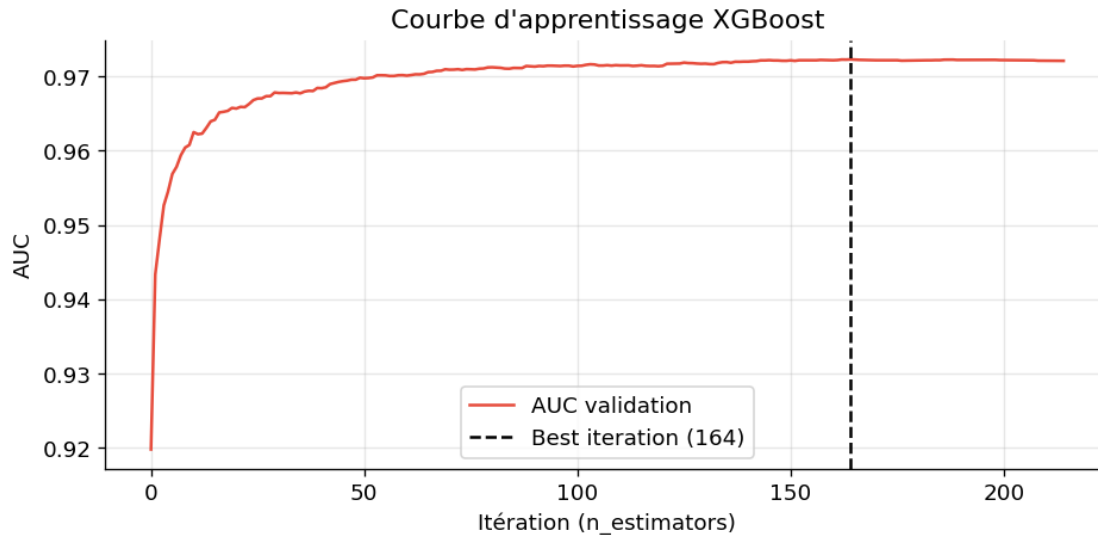
```
[20]: # Courbes ROC + PR
models_results = [
    ("Random Forest", y_proba_rf, metrics_rf),
    ("XGBoost", y_proba_xgb, metrics_xgb),
]
plot_roc_pr(models_results, y_test,
    title="Comparaison RF vs XGBoost - Classification PFAS (EPA 2024)")
```



```
[21]: # Matrices de confusion
models_preds = [
    ("Random Forest", y_pred_rf),
    ("XGBoost", y_pred_xgb),
]
plot_confusion_matrices(models_preds, y_test)
```



```
[22]: # Courbe d'apprentissage XGBoost (évolution AUC vs n_estimators)
results_history = xgb_model.evals_result()
if results_history:
    val_auc = results_history["validation_0"]["auc"]
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(val_auc, color="#e74c3c", lw=1.5, label="AUC validation")
    ax.axvline(xgb_model.best_iteration, color="k", linestyle="--",
               label=f"Best iteration ({xgb_model.best_iteration})")
    ax.set_xlabel("Itération (n_estimators)")
    ax.set_ylabel("AUC")
    ax.set_title("Courbe d'apprentissage XGBoost")
    ax.legend()
    ax.grid(alpha=0.3)
    plt.tight_layout()
    fig.savefig(FIGURES_DIR / "xgb_learning_curve.png", dpi=150,
               bbox_inches="tight")
    plt.show()
```



1.14 8bis. Gestion du déséquilibre — seuil de décision optimal

La cible EPA 2024 est quasi-équilibrée (1:1.2) et déjà compensée par `class_weight="balanced"` (RF) et `scale_pos_weight` (XGBoost). On vérifie néanmoins si le **seuil de décision** par défaut (0.50) est optimal, ou si on peut **améliorer le rappel** (détection des puits MCL) sans sacrifier la précision. Le seuil est choisi sur des probabilités **out-of-fold (train)** pour éviter toute fuite, puis appliqué au test.

```
[23]: from sklearn.model_selection import cross_val_predict

def tune_threshold(model, X_tr, y_tr, label):
    """Seuil maximisant le F1 sur probabilités out-of-fold (train) - sans fuite.
    ↪ """
    oof = cross_val_predict(model, X_tr, y_tr, cv=skf,
                           method="predict_proba", n_jobs=-1)[: , 1]
    grid = np.linspace(0.05, 0.95, 181)
    f1s = np.array([f1_score(y_tr, (oof >= t).astype(int)) for t in grid])
    recs = np.array([recall_score(y_tr, (oof >= t).astype(int)) for t in grid])
    precs = np.array([precision_score(y_tr, (oof >= t).astype(int),
    ↪ zero_division=0) for t in grid])
    t_best = grid[int(np.argmax(f1s))]
    print(f" {label:13s}: seuil F1-optimal = {t_best:.3f} (défaut 0.500)")
    return t_best, grid, f1s, recs, precs

rf_cv = RandomForestClassifier(**{**RF_PARAMS, "oob_score": False})
xgb_cv = xgb.XGBClassifier(
    **{k: v for k, v in XGB_PARAMS.items() if k not in ("eval_metric",
    ↪ "n_estimators")},
    n_estimators=int(xgb_model.best_iteration or XGB_PARAMS["n_estimators"]),
```

```

        verbosity=0,
    )

    print("Recherche du seuil optimal (out-of-fold, train)...")
    t_rf, g, f1_rf, rec_rf, pre_rf = tune_threshold(rf_cv, X_train_proc,
        ↪y_train, "Random Forest")
    t_xgb, _, f1_xgb, rec_xgb, pre_xgb = tune_threshold(xgb_cv, X_train_proc,
        ↪y_train, "XGBoost")

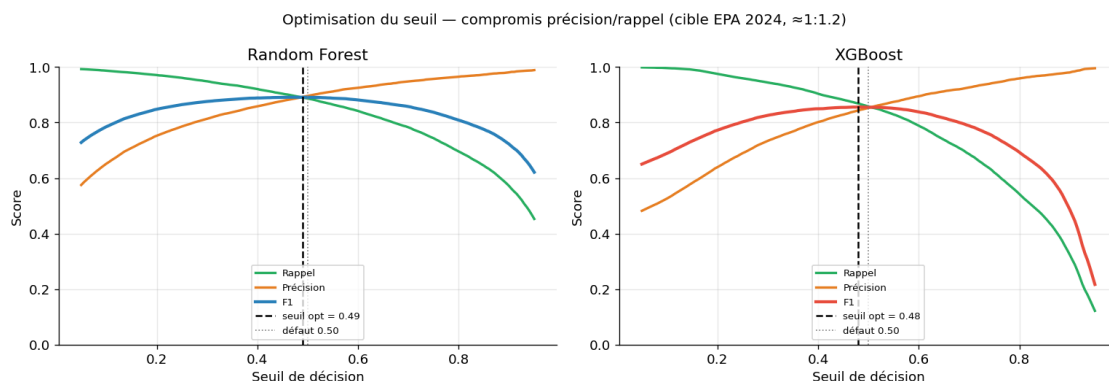
    fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
    for ax, (name, f1s, recs, precs, t_best, color) in zip(axes, [
        ("Random Forest", f1_rf, rec_rf, pre_rf, t_rf, "#2980b9"),
        ("XGBoost", f1_xgb, rec_xgb, pre_xgb, t_xgb, "#e74c3c")]):
        ax.plot(g, recs, label="Rappel", color="#27ae60", lw=2)
        ax.plot(g, precs, label="Précision", color="#e67e22", lw=2)
        ax.plot(g, f1s, label="F1", color=color, lw=2.5)
        ax.axvline(t_best, ls="--", color="k", lw=1.5, label=f"seuil opt = {t_best:.
        ↪2f}")
        ax.axvline(0.5, ls=":", color="gray", lw=1, label="défaut 0.50")
        ax.set(xlabel="Seuil de décision", ylabel="Score", title=name, ylim=(0,1))
        ax.legend(fontsize=8, loc="lower center"); ax.grid(alpha=0.3)
    fig.suptitle("Optimisation du seuil - compromis précision/rappel (cible EPA_
    ↪2024, 1:1.2)", fontsize=12)
    plt.tight_layout()
    fig.savefig(FIGURES_DIR / "ml_binary_threshold_tuning.png", dpi=150,
        ↪bbox_inches="tight")
    plt.show()

```

Recherche du seuil optimal (out-of-fold, train)...

Random Forest: seuil F1-optimal = 0.490 (défaut 0.500)

XGBoost : seuil F1-optimal = 0.480 (défaut 0.500)



```
[24]: # Effet du seuil optimal sur le TEST : défaut 0.50 vs seuil ajusté
def at_threshold(y_true, proba, t):
    yp = (proba >= t).astype(int)
    return {"seuil": t, "rappel": recall_score(y_true, yp),
            "précision": precision_score(y_true, yp, zero_division=0),
            "f1": f1_score(y_true, yp),
            "bal_acc": balanced_accuracy_score(y_true, yp)}

rows = []
for name, proba, t_best in [("Random Forest", y_proba_rf, t_rf),
                           ("XGBoost", y_proba_xgb, t_xgb)]:
    d0 = at_threshold(y_test, proba, 0.50); d0["modèle"] = name; d0["règle"] = "défaut 0.50"
    dt = at_threshold(y_test, proba, t_best); dt["modèle"] = name; dt["règle"] = f"optimal {t_best:.2f}"
    rows += [d0, dt]

thr_df = pd.DataFrame(rows)[["modèle", "règle", "seuil", "rappel", "précision", "f1", "bal_acc"]]
print("Impact du seuil sur le test set :")
for name in ["Random Forest", "XGBoost"]:
    sub = thr_df[thr_df["modèle"] == name]
    d_rec = sub.iloc[1]["rappel"] - sub.iloc[0]["rappel"]
    print(f" {name:13s}: rappel {sub.iloc[0]['rappel']:.3f} -> {sub.iloc[1]['rappel']:.3f} (Δ={d_rec:+.3f})")

OPTIMAL_THRESHOLDS = {"random_forest": float(t_rf), "xgboost": float(t_xgb)}
thr_df.style.format({"seuil": "{:.2f}", "rappel": "{:.3f}", "précision": "{:.3f}",
                    "f1": "{:.3f}", "bal_acc": "{:.3f}"})\
    .background_gradient(cmap="RdYlGn", subset=["rappel", "f1", "bal_acc"])
```

Impact du seuil sur le test set :

Random Forest: rappel 0.915 -> 0.918 (Δ=+0.003)

XGBoost : rappel 0.916 -> 0.921 (Δ=+0.006)

[24]: <pandas.io.formats.style.Styler at 0x7cd95b5c1a90>

1.15 9. Importance des features

```
[25]: # Importance MDI Random Forest
fi_rf = pd.DataFrame({
    "feature": feature_names,
    "importance": rf.feature_importances_,
}).sort_values("importance", ascending=False).reset_index(drop=True)
fi_rf["rank"] = fi_rf.index + 1

# Importance XGBoost (gain)
```

```

xgb_scores = xgb_model.get_booster().get_score(importance_type="gain")
fi_xgb = pd.DataFrame([
    {"feature": f"f{i}", "xgb_gain": xgb_scores.get(f"f{i}", 0.0)}
    for i in range(len(feature_names))
])
fi_xgb["feature"] = feature_names
fi_xgb = fi_xgb.sort_values("xgb_gain", ascending=False).reset_index(drop=True)
fi_xgb["rank_xgb"] = fi_xgb.index + 1

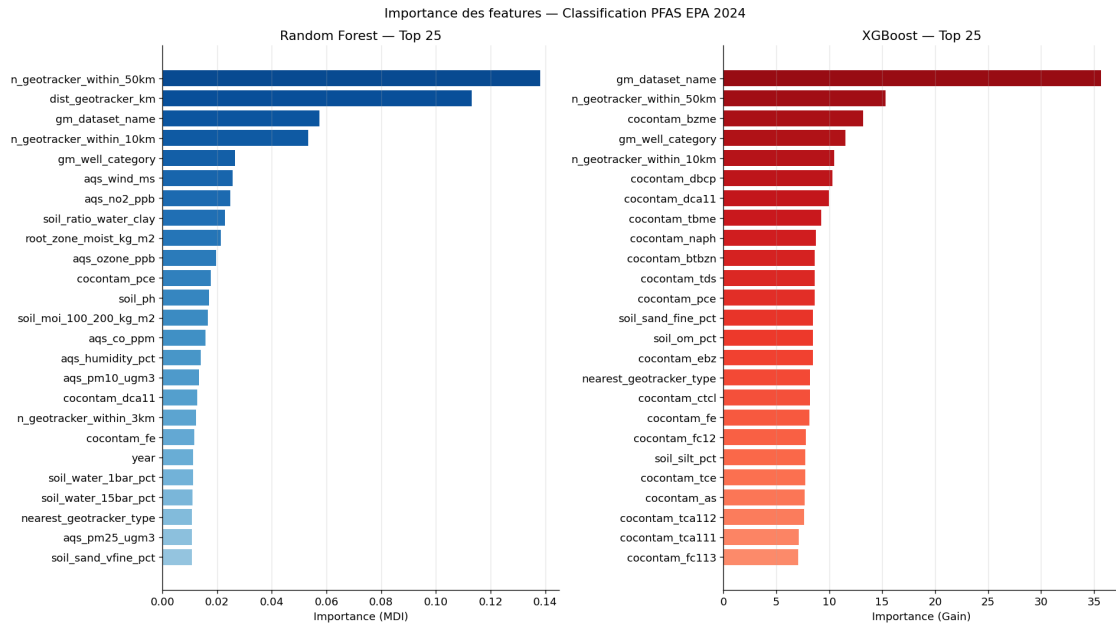
# Graphique côte à côte : Top 25
TOP_N = 25
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 9))

top_rf = fi_rf.head(TOP_N)
ax1.barh(top_rf["feature"][::-1], top_rf["importance"][::-1],
          color=plt.cm.Blues(np.linspace(0.4, 0.9, TOP_N)))
ax1.set_xlabel("Importance (MDI)")
ax1.set_title(f"Random Forest - Top {TOP_N}")
ax1.grid(axis="x", alpha=0.3)

top_xgb = fi_xgb.head(TOP_N)
ax2.barh(top_xgb["feature"][::-1], top_xgb["xgb_gain"][::-1],
          color=plt.cm.Reds(np.linspace(0.4, 0.9, TOP_N)))
ax2.set_xlabel("Importance (Gain)")
ax2.set_title(f"XGBoost - Top {TOP_N}")
ax2.grid(axis="x", alpha=0.3)

plt.suptitle(f"Importance des features - Classification PFAS EPA 2024",
             ↪fontsize=13)
plt.tight_layout()
fig.savefig(FIGURES_DIR / "ml_binary_feature_importance_comparison.png",
            ↪dpi=150, bbox_inches="tight")
plt.show()

```



```
[26]: # Concordance RF vs XGBoost (rang des 30 premières features)
top30_rf = set(fi_rf.head(30)["feature"])
top30_xgb = set(fi_xgb.head(30)["feature"])
common = top30_rf & top30_xgb
print(f"Features dans le Top 30 des deux modèles : {len(common)}/30")
print("\nFeatures communes (classées par rang moyen) :")
merged = (
    fi_rf[["feature", "rank"]]
    .merge(fi_xgb[["feature", "rank_xgb"]], on="feature")
)
merged["rank_mean"] = (merged["rank"] + merged["rank_xgb"]) / 2
merged_common = merged[merged["feature"].isin(common)].sort_values("rank_mean")
merged_common.columns = ["Feature", "Rang RF", "Rang XGB", "Rang moyen"]
merged_common.reset_index(drop=True, inplace=True)
merged_common.head(20)
```

Features dans le Top 30 des deux modèles : 13/30

Features communes (classées par rang moyen) :

```
[26]:
```

	Feature	Rang RF	Rang XGB	Rang moyen
0	n_geotracker_within_50km	1	2	1.5
1	gm_dataset_name	3	1	2.0
2	n_geotracker_within_10km	4	5	4.5
3	gm_well_category	5	4	4.5
4	cocontam_pce	11	12	11.5
5	cocontam_dca11	17	7	12.0

6	aqs_ozone_ppb	10	26	18.0
7	soil_ratio_water_clay	8	28	18.0
8	cocontam_fe	19	18	18.5
9	nearest_geotracker_type	23	16	19.5
10	soil_water_15bar_pct	22	27	24.5
11	cocontam_as	28	22	25.0
12	soil_water_1bar_pct	21	30	25.5

1.16 10. Analyse SHAP (optionnel — lent ~2 min)

```
[27]: # Mettre RUN_SHAP = True pour activer
RUN_SHAP = False

if RUN_SHAP:
    import shap
    rng = np.random.RandomState(RANDOM_STATE)
    idx_sample = rng.choice(len(X_test_proc), 1500, replace=False)
    X_sample = X_test_proc[idx_sample]

    # SHAP XGBoost
    print("Calcul SHAP XGBoost...")
    explainer_xgb = shap.TreeExplainer(xgb_model)
    shap_xgb = explainer_xgb.shap_values(X_sample)
    sv_xgb = shap_xgb if not isinstance(shap_xgb, list) else shap_xgb[1]

    fig, ax = plt.subplots(figsize=(9, 8))
    shap.summary_plot(sv_xgb, X_sample, feature_names=feature_names,
                      max_display=25, show=False, plot_size=None)
    plt.title("SHAP Summary - XGBoost (PFAS EPA 2024)", pad=12)
    plt.tight_layout()
    fig.savefig(FIGURES_DIR / "shap_summary_xgb.png", dpi=150,
↳bbox_inches="tight")
    plt.show()

    # SHAP Random Forest
    print("Calcul SHAP Random Forest...")
    explainer_rf = shap.TreeExplainer(rf)
    shap_rf = explainer_rf.shap_values(X_sample)
    sv_rf = shap_rf[1] if isinstance(shap_rf, list) else shap_rf

    fig, ax = plt.subplots(figsize=(9, 8))
    shap.summary_plot(sv_rf, X_sample, feature_names=feature_names,
                      max_display=25, show=False, plot_size=None)
    plt.title("SHAP Summary - Random Forest (PFAS EPA 2024)", pad=12)
    plt.tight_layout()
    fig.savefig(FIGURES_DIR / "shap_summary_rf.png", dpi=150,
↳bbox_inches="tight")
```

```

plt.show()

# SHAP waterfall pour un échantillon positif et un négatif
pos_idx = np.where(y_test[idx_sample] == 1)[0][0]
neg_idx = np.where(y_test[idx_sample] == 0)[0][0]
for label, sample_idx in [("positif ( MCL)", pos_idx), ("négatif (< MCL)",
neg_idx)]:
    shap.waterfall_plot(
        shap.Explanation(values=sv_xgb[sample_idx],
                        base_values=explainer_xgb.expected_value,
                        data=X_sample[sample_idx],
                        feature_names=feature_names),
        max_display=15, show=False
    )
    plt.title(f"SHAP Waterfall - XGBoost, échantillon {label}")
    plt.tight_layout()
    plt.savefig(FIGURES_DIR / f"shap_waterfall_xgb_{label.split()[0]}.png",
                dpi=150, bbox_inches="tight")
    plt.show()
else:
    print("SHAP désactivé. Mettre RUN_SHAP = True pour l'activer.")

```

SHAP désactivé. Mettre RUN_SHAP = True pour l'activer.

1.17 10ter. Visualisations de synthèse (soutenance)

Graphiques à fort impact pour la présentation devant le jury : 1. **Carte des prédictions** — résultat spatial (bien que la localisation ne soit pas une feature) 2. **Courbe de calibration** — fiabilité des probabilités prédites 3. **Courbe de gain cumulé** — valeur opérationnelle : priorisation de l'échantillonnage 4. **Tableau de bord comparatif** — RF vs XGBoost sur toutes les métriques

```

[28]: # Carte des prédictions sur la Californie (test set, modèle Random Forest)
coords = df.loc[X_test.index, ["longitude", "latitude"]].copy()
coords["y_true"] = y_test
coords["proba"] = y_proba_rf
thr_rf = OPTIMAL_THRESHOLDS["random_forest"]
coords["y_pred"] = (coords["proba"].values >= thr_rf).astype(int)

fig, axes = plt.subplots(1, 3, figsize=(18, 7))

# (1) Réalité terrain
for lab, c, m in [(0, "#3498db", "< MCL"), (1, "#e74c3c", " MCL")]:
    sub = coords[coords["y_true"] == lab]
    axes[0].scatter(sub["longitude"], sub["latitude"], s=7, c=c, alpha=0.45,
                    label=m, edgecolors="none")
axes[0].set_title("Réalité terrain (test set)")
axes[0].legend(markerscale=2, loc="upper right")

```

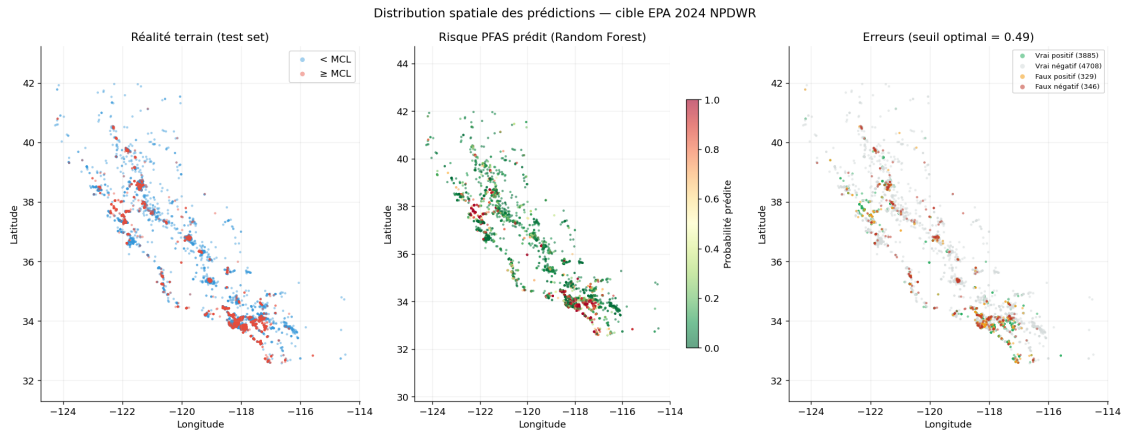
```

# (2) Probabilité prédite
sc = axes[1].scatter(coords["longitude"], coords["latitude"], s=7,
    ↪c=coords["proba"],
    cmap="RdYlGn_r", alpha=0.6, vmin=0, vmax=1,
    ↪edgecolors="none")
plt.colorbar(sc, ax=axes[1], shrink=0.7, label="Probabilité prédite")
axes[1].set_title("Risque PFAS prédit (Random Forest)")

# (3) Type d'erreur de classification
conds = [
    (coords.y_true == 1) & (coords.y_pred == 1),
    (coords.y_true == 0) & (coords.y_pred == 0),
    (coords.y_true == 0) & (coords.y_pred == 1),
    (coords.y_true == 1) & (coords.y_pred == 0),
]
labels = ["Vrai positif", "Vrai négatif", "Faux positif", "Faux négatif"]
import numpy as np
coords["err"] = np.select(conds, labels, default="?")
palette = {"Vrai positif": "#27ae60", "Vrai négatif": "#d5dbdb",
    "Faux positif": "#f39c12", "Faux négatif": "#c0392b"}
for cat, c in palette.items():
    sub = coords[coords["err"] == cat]
    axes[2].scatter(sub["longitude"], sub["latitude"], s=7, c=c, alpha=0.55,
        label=f"{cat} ({len(sub)}", edgecolors="none")
axes[2].set_title(f"Erreurs (seuil optimal = {thr_rf:.2f})")
axes[2].legend(markerscale=2, fontsize=8, loc="upper right")

for ax in axes:
    ax.set_xlabel("Longitude"); ax.set_ylabel("Latitude")
    ax.set_aspect("equal", adjustable="datalim"); ax.grid(alpha=0.2)
fig.suptitle("Distribution spatiale des prédictions - cible EPA 2024 NPDWR",
    ↪fontsize=14)
plt.tight_layout()
fig.savefig(FIGURES_DIR / "ml_binary_map_predictions.png", dpi=150,
    ↪bbox_inches="tight")
plt.show()

```



```
[29]: # Courbe de calibration : les probabilités prédites sont-elles fiables ?
from sklearn.calibration import calibration_curve

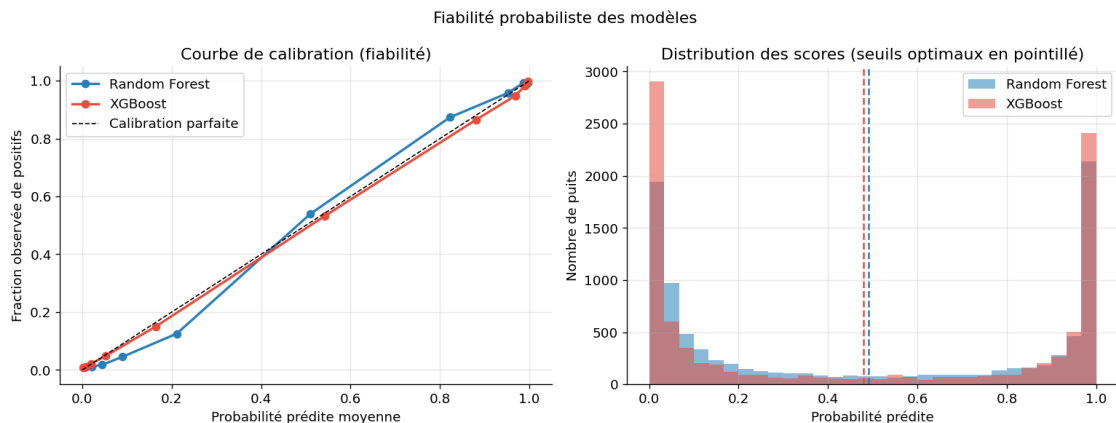
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5),
                              gridspec_kw={"height_ratios": [1]})

for name, proba, color in [("Random Forest", y_proba_rf, "#2980b9"),
                           ("XGBoost", y_proba_xgb, "#e74c3c")]:
    frac_pos, mean_pred = calibration_curve(y_test, proba, n_bins=10,
    ↪ strategy="quantile")
    ax1.plot(mean_pred, frac_pos, "o-", color=color, lw=2, label=name)
    ax1.plot([0, 1], [0, 1], "k--", lw=1, label="Calibration parfaite")
    ax1.set(xlabel="Probabilité prédite moyenne", ylabel="Fraction observée de
    ↪ positifs",
            title="Courbe de calibration (fiabilité)")
    ax1.legend(); ax1.grid(alpha=0.3)

# Histogramme des probabilités prédites
ax2.hist(y_proba_rf, bins=30, alpha=0.55, color="#2980b9", label="Random
    ↪ Forest")
ax2.hist(y_proba_xgb, bins=30, alpha=0.55, color="#e74c3c", label="XGBoost")
ax2.axvline(OPTIMAL_THRESHOLDS["random_forest"], ls="--", color="#2980b9", lw=1.
    ↪ 5)
ax2.axvline(OPTIMAL_THRESHOLDS["xgboost"], ls="--", color="#e74c3c", lw=1.5)
ax2.set(xlabel="Probabilité prédite", ylabel="Nombre de puits",
        title="Distribution des scores (seuils optimaux en pointillé)")
ax2.legend(); ax2.grid(alpha=0.3)

fig.suptitle("Fiabilité probabiliste des modèles", fontsize=13)
plt.tight_layout()
fig.savefig(FIGURES_DIR / "ml_binary_calibration.png", dpi=150,
    ↪ bbox_inches="tight")
```

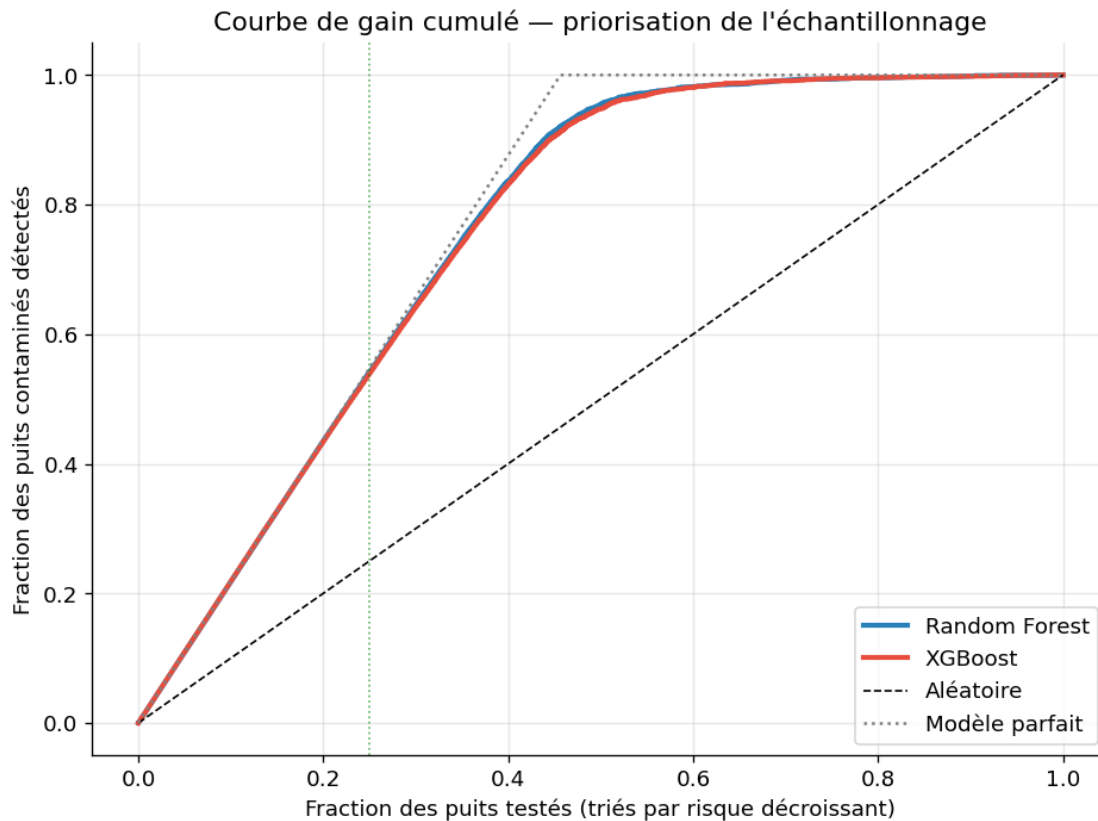
```
plt.show()
```



```
[30]: # Courbe de gain cumulé : valeur opérationnelle de la priorisation
def cumulative_gain(y_true, proba):
    order = np.argsort(proba)[::-1]
    y_sorted = np.asarray(y_true)[order]
    gains = np.cumsum(y_sorted) / y_sorted.sum()
    frac = np.arange(1, len(y_sorted) + 1) / len(y_sorted)
    return np.concatenate([[0], frac]), np.concatenate([[0], gains])

prev = y_test.mean()
fig, ax = plt.subplots(figsize=(8, 6))
annot = []
for name, proba, color in [("Random Forest", y_proba_rf, "#2980b9"),
                           ("XGBoost", y_proba_xgb, "#e74c3c")]:
    frac, gains = cumulative_gain(y_test, proba)
    ax.plot(frac, gains, color=color, lw=2.5, label=name)
    # gain à 25% des puits testés
    g25 = np.interp(0.25, frac, gains)
    annot.append((name, g25))
ax.plot([0, 1], [0, 1], "k--", lw=1, label="Aléatoire")
ax.plot([0, prev, 1], [0, 1, 1], color="gray", ls=":", lw=1.5, label="Modèle_
↳parfait")
ax.axvline(0.25, color="green", ls=":", lw=1, alpha=0.6)
ax.set(xlabel="Fraction des puits testés (triés par risque décroissant)",
       ylabel="Fraction des puits contaminés détectés",
       title="Courbe de gain cumulé - priorisation de l'échantillonnage")
ax.legend(loc="lower right"); ax.grid(alpha=0.3)
plt.tight_layout()
fig.savefig(FIGURES_DIR / "ml_binary_cumulative_gain.png", dpi=150,
↳bbox_inches="tight")
plt.show()
```

```
print("Lecture opérationnelle (en testant les 25% puits les plus à risque) :")
for name, g25 in annot:
    print(f" {name:13s}: on détecte {100*g25:.0f}% des puits réellement_
    ↪contaminés")
```



Lecture opérationnelle (en testant les 25% puits les plus à risque) :

Random Forest: on détecte 54% des puits réellement contaminés

XGBoost : on détecte 54% des puits réellement contaminés

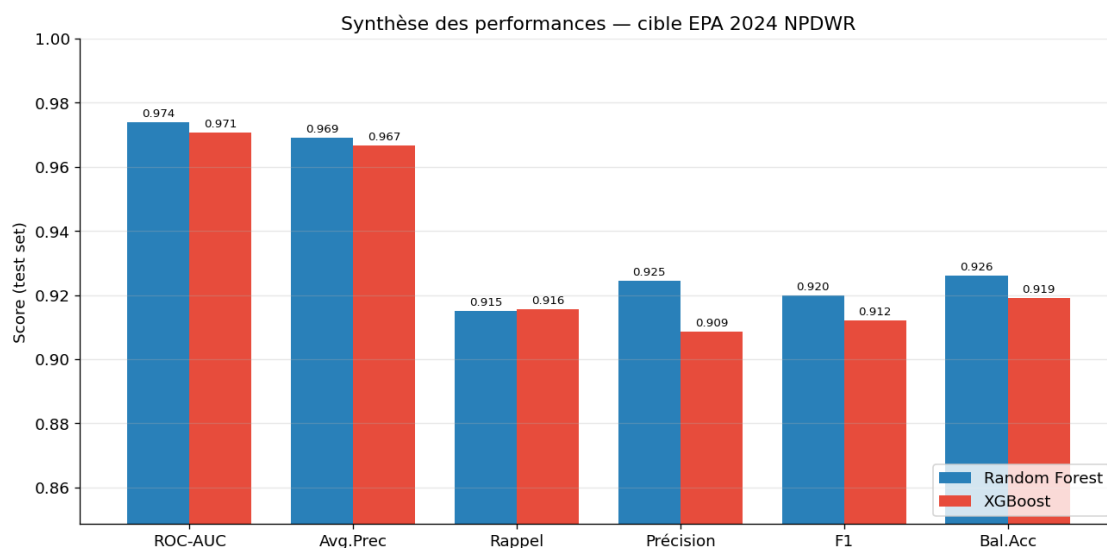
```
[31]: # Tableau de bord comparatif RF vs XGBoost (toutes métriques)
keys = ["roc_auc", "avg_precision", "recall", "precision", "f1",
    ↪"balanced_accuracy"]
labels = ["ROC-AUC", "Avg.Prec", "Rappel", "Précision", "F1", "Bal.Acc"]
rf_vals = [metrics_rf[k] for k in keys]
xgb_vals = [metrics_xgb[k] for k in keys]

x = np.arange(len(labels)); w = 0.38
fig, ax = plt.subplots(figsize=(11, 5.5))
b1 = ax.bar(x - w/2, rf_vals, w, label="Random Forest", color="#2980b9")
b2 = ax.bar(x + w/2, xgb_vals, w, label="XGBoost", color="#e74c3c")
```

```

ax.bar_label(b1, fmt="%.3f", fontsize=8, padding=2)
ax.bar_label(b2, fmt="%.3f", fontsize=8, padding=2)
ymin = max(0.0, min(rf_vals + xgb_vals) - 0.06)
ax.set_xticks(x); ax.set_xticklabels(labels)
ax.set_ylim(ymin, 1.0)
ax.set_ylabel("Score (test set)")
ax.set_title("Synthèse des performances - cible EPA 2024 NPDWR")
ax.legend(loc="lower right"); ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
fig.savefig(FIGURES_DIR / "ml_binary_dashboard.png", dpi=150,
    ↳bbox_inches="tight")
plt.show()

```



1.18 11. Sauvegarde des artefacts

```

[32]: # Feature importance CSV
fi_combined = fi_rf.rename(columns={"importance": "rf_mdi", "rank": "rank_rf"}).
    ↳merge(
        fi_xgb[["feature", "xgb_gain", "rank_xgb"]], on="feature", how="left"
    ).sort_values("rf_mdi", ascending=False)
fi_combined.to_csv(PROCESSED_DIR / "ml_binary_feature_importance.csv",
    ↳index=False)
print(f"Feature importance → {PROCESSED_DIR / 'ml_binary_feature_importance.'
    ↳csv}")

# JSON résultats
results = {
    "target": "target_epa2024",

```

```

"regulation": "EPA 2024 NPDWR (MCLs individuels + HI mélange)",
"mcls": EPA2024_MCLS,
"dataset": {"n_total": len(df), "n_features": len(feature_names),
            "n_pos": int(y.sum()), "pct_pos": round(100*y.mean(), 2)},
"split": {"train": int(len(y_train)), "test": int(len(y_test))},
"optimal_thresholds": OPTIMAL_THRESHOLDS,
"random_forest": {
    "params": {k: v for k, v in RF_PARAMS.items() if k not in ("n_jobs",)},
    "oob_score": round(rf.oob_score_, 4),
    "test": {k: round(v, 4) for k, v in metrics_rf.items() if k != "name"},
    "cv_roc_auc": f"{cv_rf['test_roc_auc'].mean():.
↪4f}±{cv_rf['test_roc_auc'].std():.4f}",
    "cv_f1":      f"{cv_rf['test_f1'].mean():.4f}±{cv_rf['test_f1'].std():.
↪4f}",
    "cv_recall":  f"{cv_rf['test_recall'].mean():.4f}±{cv_rf['test_recall'].
↪std():.4f}",
    "cv_balanced_accuracy": f"{cv_rf['test_balanced_accuracy'].mean():.
↪4f}±{cv_rf['test_balanced_accuracy'].std():.4f}",
},
"xgboost": {
    "best_iteration": int(xgb_model.best_iteration),
    "best_val_auc": round(float(xgb_model.best_score), 4),
    "test": {k: round(v, 4) for k, v in metrics_xgb.items() if k != "name"},
    "cv_roc_auc": f"{cv_xgb['test_roc_auc'].mean():.
↪4f}±{cv_xgb['test_roc_auc'].std():.4f}",
    "cv_f1":      f"{cv_xgb['test_f1'].mean():.4f}±{cv_xgb['test_f1'].std():
↪.4f}",
    "cv_recall":  f"{cv_xgb['test_recall'].mean():.
↪4f}±{cv_xgb['test_recall'].std():.4f}",
    "cv_balanced_accuracy": f"{cv_xgb['test_balanced_accuracy'].mean():.
↪4f}±{cv_xgb['test_balanced_accuracy'].std():.4f}",
},
"top10_features_rf": fi_rf.head(10)[["rank", "feature", "importance"]].
↪rename(columns={"importance": "rf_mdi"}).to_dict("records"),
"top10_features_xgb": fi_xgb.head(10)[["rank_xgb", "feature", "xgb_gain"]].
↪to_dict("records"),
}
with open(PROCESSED_DIR / "ml_binary_results.json", "w") as f:
    json.dump(results, f, indent=2)
print(f"Résultats JSON → {PROCESSED_DIR / 'ml_binary_results.json'}")

# Modèles pickle
artifact_rf = {
    "model": rf, "preprocessor": preprocessor,
    "optimal_threshold": OPTIMAL_THRESHOLDS["random_forest"],
    "feature_names": feature_names, "num_cols": num_cols, "cat_cols": cat_cols,

```



```

        "target": "target_epa2024", "epa2024_mcls": EPA2024_MCLS,
    }
    with open(MODELS_DIR / "rf_binary_epa2024.pkl", "wb") as f:
        pickle.dump(artifact_rf, f)
    print(f"RF → {MODELS_DIR / 'rf_binary_epa2024.pkl'}")

    artifact_xgb = {
        "model": xgb_model, "preprocessor": preprocessor,
        "optimal_threshold": OPTIMAL_THRESHOLDS["xgboost"],
        "feature_names": feature_names, "num_cols": num_cols, "cat_cols": cat_cols,
        "target": "target_epa2024", "epa2024_mcls": EPA2024_MCLS,
    }
    with open(MODELS_DIR / "xgb_binary_epa2024.pkl", "wb") as f:
        pickle.dump(artifact_xgb, f)
    print(f"XGB → {MODELS_DIR / 'xgb_binary_epa2024.pkl'}")

```

Feature importance → ../data/processed/ml_binary_feature_importance.csv

Résultats JSON → ../data/processed/ml_binary_results.json

RF → ../models/rf_binary_epa2024.pkl

XGB → ../models/xgb_binary_epa2024.pkl

1.19 12. Synthèse finale

```

[33]: print("
print("      RÉSULTATS - Classification binaire PFAS      ")
print("      Réglementation : EPA 2024 NPDWR      ")
print("
print(f"  Dataset : {len(df):,} échantillons | {len(feature_names)} features_
↪      ")
print(f"  Positifs : {int(y.sum()):,} ({100*y.mean():.1f}%)  Négatifs :_
↪ {int((y==0).sum()):,} ({100*(y==0).mean():.1f}%)  ")
print("
print(f"  {'Modèle':<20} {'AUC':>7} {'AP':>7} {'F1':>7}      ")
print("
for name, m in [("Random Forest", metrics_rf), ("XGBoost", metrics_xgb)]:
    print(f"  {name:<20} {m['roc_auc']:>7.4f} {m['avg_precision']:>7.4f}_
↪ {m['f1']:>7.4f}      ")
print("
print("  Top 5 features communes (RF  XGBoost) :      ")
for _, row in merged_common.head(5).iterrows():
    feat = str(row["Feature"])[38]
    print(f"      #{row['Rang RF']:3.0f}/{row['Rang XGB']:3.0f}  {feat:<38} ")
print("

```

RÉSULTATS - Classification binaire PFAS

Réglementation : EPA 2024 NPDWR

Dataset : 46,338 échantillons | 86 features
Positifs : 21,154 (45.7%) Négatifs : 25,184 (54.3%)

Modèle	AUC	AP	F1
Random Forest	0.9740	0.9690	0.9198
XGBoost	0.9708	0.9666	0.9121

Top 5 features communes (RF XGBoost) :

- # 1/ 2 n_geotracker_within_50km
- # 3/ 1 gm_dataset_name
- # 4/ 5 n_geotracker_within_10km
- # 5/ 4 gm_well_category
- # 11/ 12 cocontam_pce